

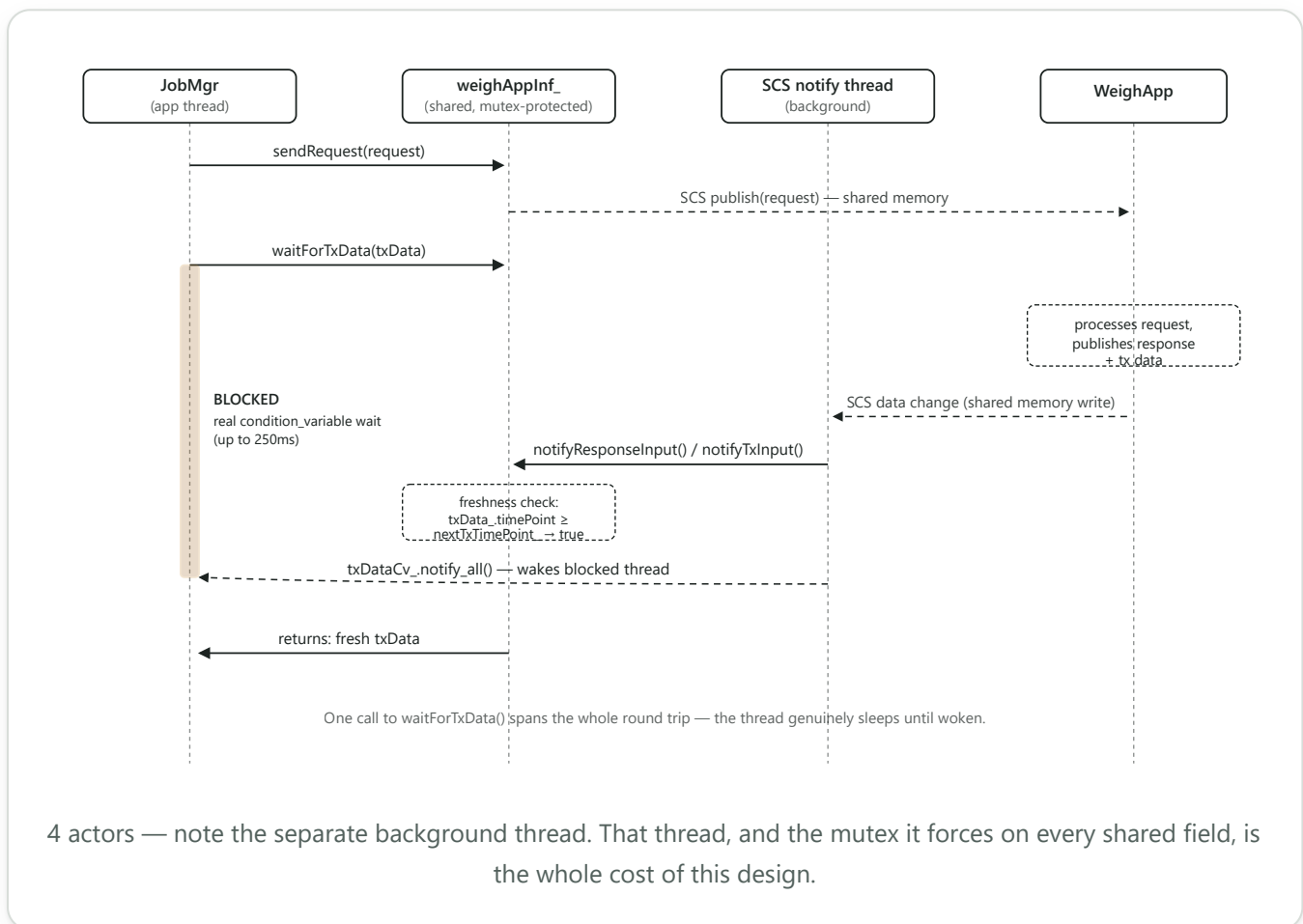
## COMPANION TO THE MIGRATION BRIEFING

## Request/response flow — before and after

Stage 1 scope: JobMgr sends a request to WeighApp and waits for fresh weighing data back. Same operation, two architectures.

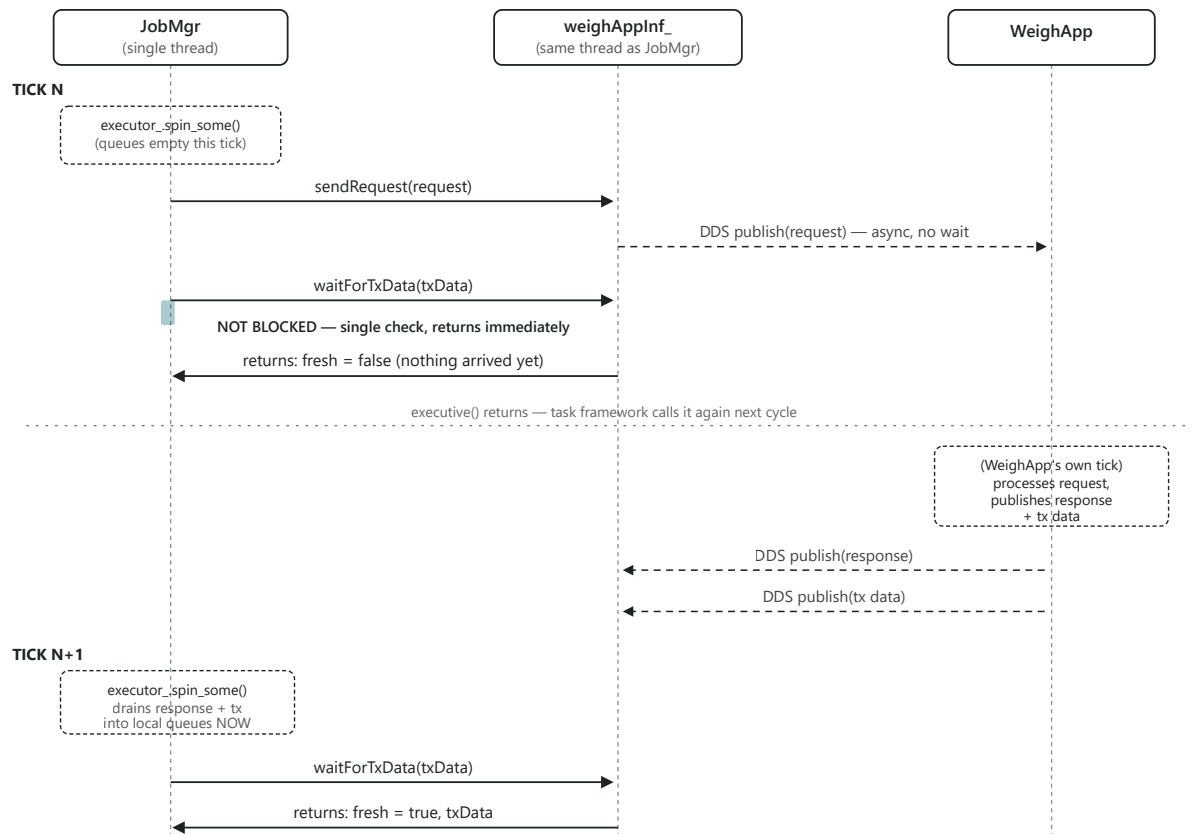
**Before: real SCS** CALLBACK THREAD

A background thread — separate from JobMgr's own task loop — wakes JobMgr up the instant fresh data lands. This is what a real `addNewDataSlot()` callback and a genuine `condition_variable` block buy you.

**After: ROS2/DDS** POLL-BASED, SINGLE THREAD

No background thread ever touches JobMgr's own code or state directly. (DDS's own middleware likely runs internal threads for network I/O — that's normal and outside our control — but they never call into our callback except through `spin_some()`, on our own thread.) "Waiting" is really "check what already arrived, once, right now" — repeated automatically because `executive()` runs again next tick regardless. driven

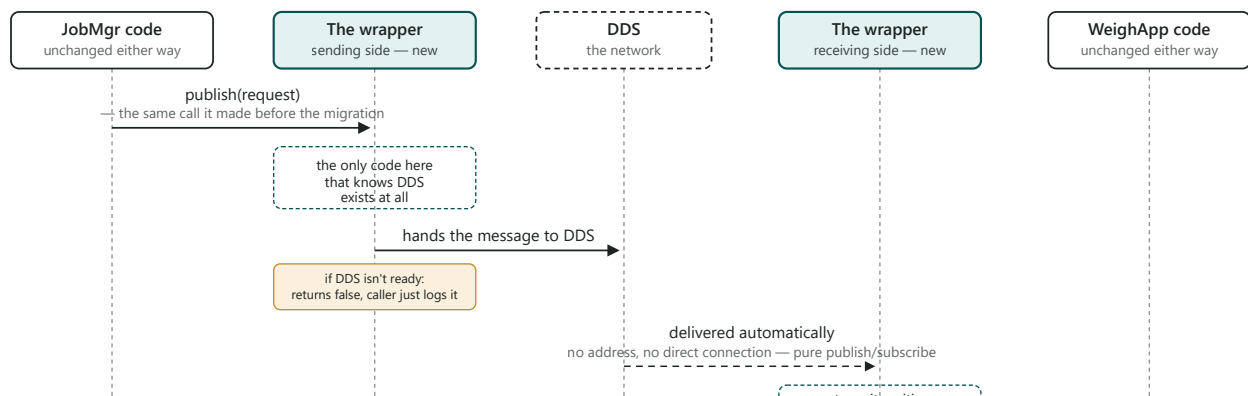
by the same task scheduler that always called it, unchanged by this migration.  
Nobody wakes JobMgr up — it was never asleep.

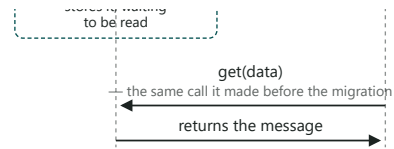


3 actors, none of them a thread that reaches into JobMgr's code. The "wait" is really two ordinary calls, one tick apart, driven by JobMgr's own pre-existing loop — nothing here ever sleeps or gets woken.

## The approach, zoomed in: what the wrapper actually does

**One sentence version:** the wrapper is the *only* piece of code in this entire system that knows ROS2/DDS exists. Everything on either side of it — JobMgr's logic, WeighApp's logic — calls the exact same `publish()` / `get()` it always called. The wrapper is where the actual swap happens; nothing else had to change to make room for it.





Both ends of the conversation — JobMgr's logic and WeighApp's logic — are exactly as they were. The entire migration lives inside the two highlighted boxes.

## What actually changed

### BEFORE — SCS

- 4 real actors, including a background notify thread
- `waitForTxData()` can genuinely block for up to 250ms
- Needs a mutex + `condition_variable` — real shared state, real race to guard
- One call captures the whole round trip

### AFTER — ROS2/DDS (POLL-BASED)

- No background thread ever calls into JobMgr's own code — DDS's internal threads, if any, only reach us through `spin_some()`
- `waitForTxData()` always returns immediately, fresh or not
- No mutex needed — our callback and our drain never run concurrently, by construction
- The round trip is spread across ordinary tick-to-tick calls instead

